

23COA202: Embedded Systems Programming

Coursework specification

Dr Mohamad Saada

Semester 1 2023-24

1 Introduction

This coursework forms 100% of the assessment for this module. There are several parts to the coursework. A main exercise (known as BASIC) and several additional parts to allow you to demonstrate more knowledge (EXTENSIONS). These are all detailed below. If anything is not clear as you read, please read through to the end of the document, it may be clear later. If still not, then post the question into the student support forum on LEARN.

2 Parking management system

The aim of this exercise is to program the Arduino as a parking management system. It will read custom input messages over the Serial Interface, process them, and then display the result on the screen. The Arduino's buttons will allow for interaction with the information displayed on the LCD. First, we will introduce the concepts of custom messages and vehicle information, then how things should be displayed on the Arduino Screen and the protocol that will be implemented across the Serial Monitor, with some examples.

2.1 Serial messages and vehicle information

The car park management system receives communication over the Serial Interface in a series of custom messages. These are defined as follows:

- After an initialisation phase, each message is a text string sent as a sequence of characters terminated by a "Newline" or "Carriage return" control character. These are sent over the serial interface either through the Serial Monitor or from a separate host program.
 - "Carriage return" or "CR" programmatically written as "\r" is a control character that makes the printing cursor go to the beginning of the line. This is sent from Python with a `print('\r')` or from the Serial monitor by selecting "Carriage return" from the drop-down at the bottom of the window.

- “Newline” or “NL” or “Line Feed” or “LF” programmatically written as “\n” is a control character that makes the printing cursor go one line down. This is sent from Python with a `print('\n')` or from the Serial monitor by selecting “Newline” from the drop-down at the bottom of the window.
- Vehicles are represented by a registration numbers, which are made of 7 alphanumeric characters in a specific order, where it starts with 2 letters in the range A-Z, followed by 2 numbers in the range 0-9 followed by another 3 letters in the ranges A-Z spaces are not allowed.

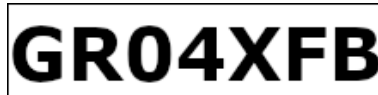


Figure 1: Vehicle registration number

- Each vehicle has a single letter denoting its type which can be 1 of 5 different vehicle types (i.e. C, M, V, L, B), these are car, motorcycle, van, lorry or bus.
- Each Vehicle has a location text to indicate the name of the parking lot, which is a string between 1-11 characters long from the range A-Z and 0-9 and . (period) (e.g. GranbySt., BeehiveLane, BrownsLane, etc.), spaces are not allowed, and location string cannot be empty.
- Each Vehicle has an entry timestamp to indicate when it has entered the parking lot, which is of the format HHMM spaces are not allowed, and entry timestamp cannot be empty, and is assigned automatically upon receipt of entry message.
- Vehicle that have paid the parking fee have an exit timestamp to indicate when the fee was paid, which is of the format HHMM spaces are not allowed, and exit timestamp can be empty while the vehicle is still parked, and is assigned automatically upon receipt of payment message.
- Each vehicle has an indicator for whether a payment has been made for parking (i.e. Paid or Not Paid). These values are provided across the serial monitor as PD or NPD, and special display colours should be used depending on the state of payment.

The protocol for communication is detailed later in the document, but first look at what is to be displayed.

2.2 Arduino Display

The Arduino presents all the information of all the vehicle that parked in its parking lots. This information is displayed on its screen and provides an interface via the buttons for the user to examine them. We describe this interface in this section.

- The display has two lines and shows the information of a single vehicle providing an up-down scrolling facility to see others.
- The display must be of the form:

```
+-----+
| ^ABCDEFG HIJKLMN | OPQR
| vS TUV HHMM HHMM |
+-----+
```

Note the ^ and v are arrow indicators for up and down. Also, there is one space between vehicle registration number (ABCDEFG) and parking location string (HIJKLMN), there is one space between vehicle type (S) and payment status (TUV), and there is one space between payment status (TUV) and entry timestamp percentage (HHMM) and one space between entry timestamp and exit timestamp (HHMM).

Note the exact positioning of the characters.

- In this example, ABCDEFG are replaced by the vehicle's registration number, HIJKLMN are replaced by the parking's location string. It is worth noting that the screen is 16 characters wide, therefore only a maximum of 7 characters from the location string will be displayed on screen.
- S is replaced by the vehicle's type (i.e. C, M, V, L, B) for car, motorcycle, van, lorry or bus.
- TUV is replaced by the vehicle's payment status (i.e. PD for Paid or NPD for Not Paid). It is worth noting that if the state is PD you would have to make sure a leading space is added to keep same displayed indicators positioning.
- If the status of payment for a vehicle that is currently displayed on the LCD screen is NPD (i.e. Not Paid), then the backlight must change to the colour: yellow, If the status of payment for a vehicle that is currently displayed on the LCD screen is PD (i.e. Paid), then the backlight must change to the colour; green.
- Program the UP and DOWN buttons to move up or down through the vehicles list in time of arrival order. On reaching the first or last vehicle remove the ^ or v, to indicate there are no more vehicles to display.
- The first HHMM is replaced by the time a vehicle enters the parking, the second HHMM is replaced by the time a vehicle pays the parking fee.
- Pressing and holding the SELECT button for longer than one second clears the screen, sets the backlight to purple and simply displays your student ID number and nothing else. Releasing it returns the display to normal. During this time, your program must continue to read input messages from the serial interface and process them.

2.3 The protocol in more detail

- Synchronisation phase:
 - After your Arduino has booted it must set the backlight to purple and repeatedly send the character Q with no following “Newline” or “Carriage return” control characters to the Serial interface at a frequency of once per second.
 - After sending, the Arduino must monitor for an incoming character and when an X is received synchronisation completes and the main program loop can start up.
 - No “Newline” or “Carriage return” characters should be sent. If a “Newline” or a “Carriage return” character is received it must be treated as an error.
- After synchronisation happens your program must send the string BASIC followed by a “Newline”, then set the backlight to white and start the main phase.
- Main phase:
 - The host sends messages to the Arduino over the serial interface. The messages will start with a character that denotes the type of requested operation, this is followed by other information, these operation characters are as follows:
 - * **A** — this indicates adding a new vehicle that entered. It will be followed by ‘-’ then seven alphanumeric characters for a vehicles reg number, then ‘-’ and then followed by a letter for the vehicles’s type (i.e. C, M, V, L, B), these are car, motorcycle, van, lorry or bus. This is followed by another ‘-’ which is followed by a String for the parking’s location. The location string could be between 1 and 11 characters from the range A-Z and 0-9 and . (period) (e.g. GranbySt., BeehiveLane, BrownsLane, etc.), and any extra characters until the “Newline” character must be treated as an error. If the host sends an “A” message for an existing vehicle reg number, then the new vehicle type and/or the new location must be used, only if the vehicle payment status is PD (i.e. Paid), otherwise if payment status is NPD (i.e. Not Paid) an error message should indicate can't be modified because of non payment status.
 - * **S** — this indicates a payment status for the vehicle (i.e. Paid or Not Paid). It will be followed by a ‘-’ then seven alphanumeric characters for a vehicles reg number, then ‘-’, that is followed by either an PD or NPD text. Vehicle payment status values received for vehicles that have not been added must be treated as an error. It is worth noting that the default status of a vehicle when added is NPD. When a vehicle status has changed to PD (i.e. Paid), the time of payment (i.e. receipt of S-REGNUMB-PD message) should be saved and displayed. If a vehicle's payment status changes from PD to NPD (i.e. Paid to Not Paid) then the payment timestamp is removed, and the entry

timestamp is changed to the time of current message receipt (i.e. receipt of S-REGNUMB-NPD message). If an S message is received which does not change the payment status (i.e. S-REGNUMB-PD when status is PD or S-REGNUMB-NPD when status is NPD), it should raise an error to indicate that payment status not changed.

- * **T** — this indicates changing the vehicle's type. It will be followed by '-' then 7 alphanumeric characters for a vehicles reg number, then '-' and then followed by a letter for the vehicle's new type (i.e. C, M, V, L, B), these are car, motorcycle, van, lorry or bus. If the host sends a "T" message with the same type of the existing vehicle reg number, an error message should indicate can't be modified because of the same vehicle type. If the host sends a "T" message and the payment status is NPD (i.e. Not Paid) an error message should indicate it can't be modified because of non payment status.
- * **L** — this indicates changing the vehicle's parking location. It will be followed by '-' then seven alphanumeric characters for a vehicles reg number, then '-' and then followed by a String for the parking's location. The location string could be between 1 and 11 characters long from the range A-Z and 0-9 and . (period) (e.g. GranbySt., BeehiveLane, BrownsLane, etc.), and any extra characters until the "Newline" character must be treated as an error. If the host sends a "L" message with the same location string of the existing vehicle location, an error message should indicate can't be modified because of the vehicle location string is the same. If the host sends a "L" message and the payment status is NPD (i.e. Not Paid) an error message should indicate it can't be modified because of non payment status.
- * **R** — this indicates removing an already existing vehicle. It will be followed by '-' then up to seven alphanumeric characters for a vehicle reg number. If the vehicle reg number does not exist, then it should be treated as an error. Otherwise, the vehicle with the matching reg number, should be deleted with all its data if its payment status is PD (i.e. Paid), otherwise an error message should indicate can't be deleted because of non payment status.
- *
 - Any lines not conforming to the protocol must output an error message of the form "ERROR: followed by the error message" should be written to the Serial Monitor. This will aid debugging. This should have no other effect on the system.
 - Lines for any operation (i.e. A, S, T, L or R) with no REGNUMB (i.e. vehicle reg number) are invalid and treated as an error.
 - Lines for A missing the vehicle type or/and parking location string fields are invalid and treated as an error, with an error message.
 - Lines for S missing the status field are invalid and treated as an error, with an error message.
 - Lines for T missing the vehicle type are invalid and treated as an error, with an error message.
 - Lines for L missing the vehicles parking location string field are invalid and treated as an error,

with an error message.

- Lines with the wrong number and/or wrong position of field separators '-' are invalid and treated as an error.
 - * In addition, you may send any text from your program beginning with **DEBUG:** to the Serial Monitor; it will be ignored by the assessment testing program but will help your debugging.
 - * You can send a message from your program with **DONE!** every time the program finishes processing a serial command.
 - * There must be no space characters in the serial command messages.
 - * Your Arduino program should not output any serial messages that don't start with **ERROR:** or **DEBUG:** or the message **DONE!** This will help the testing program understand what you are sending.
- Unless you implement the EEPROM extension (see below), then on initialisation no vehicle information should be assumed added and consequently after synchronisation all received messages not starting with A must be treated as an error until a suitable A message arrives.
 - Given that the vehicle reg numbers are made of 7 alphanumeric characters, this means that theoretically you can have $26^5 * 10^2$ number of possible vehicle reg numbers. Since the Arduino's SRAM memory is limited, please choose a reasonable enforced limit on the number of vehicles that can be added (i.e. once reached you can't add more vehicles until you delete some of the existing ones). The main goal is that your program doesn't run out of memory, yet you maximise the number of vehicles and their information that you can add. Being able to add the most amount of vehicles possible without running out of memory will be looked at advantageously.
 - When the Arduino boots up, the time and date will be 00:00 01/01/2023. If you do implement the EEPROM extension, save the current time to EEPROM every time a processed serial message causes any change.

2.4 Protocol Examples

Example uses of the protocol: The example below adds vehicles and manipulates them through different serial commands.

Example messages with teal comments are valid and get processed. Whereas messages with gold comments are not valid and provoke an error response from your program. Assume the example messages are run in this order (i.e. if a vehicle is added in one command then assume it exists for consecutive commands).

A-GR04XFB-C-GranbySt.	← Adds a vehicle with the reg number “GR04XFB”, type car and parking location “GranbySt.”
A-GF57XWD-B-BeehiveLane	← Adds a vehicle with the reg number “GF57XWD”, type bus and parking location “BeehiveLane”
A-BD51SMR-V-BrownsLane	← Adds a vehicle with the reg number “BD51SMR”, type van and parking location “BrownsLane”
S-GR04XFB-PD	← Sets vehicle with reg number “GR04XFB” to PD (i.e. Paid) payment status.
T-GR04XFB-B	← Changes type of vehicle with reg number “GR04XFB” to bus
L-GF57XWD-Southfield	← Produces an error as trying to change parking location of vehicle with reg number “GF57XWD” but its payment status is NPD
S-GF57XWD-1	← Produces an error as 1 is not a valid option for payment status
L-CU57ABC-BeehiveLane	← Produces an error as a vehicle with the reg number “CU57ABC” does not exist
L-GR04XFB-BrownsLane	← Changes parking location of vehicle with reg number “GR04XFB” to “BrownsLane”
S-BD51SMR-PD	← Sets vehicle with reg number “BD51SMR” to PD (i.e. Paid) payment status.

L-BD51SMR-BrownsLane	← Produces an error as trying to change the parking location of “BD51SMR” to the same saved parking location “BrownsLane”
A-CU57ABC-C-BedfordSquare	← Produces an error as adding a vehicle with a parking location string longer than 11 characters
A-CU57ABC-BedfordSq.	← Produces an error adding vehicle with no type
A-CU57ABC-C-BedfordSquare	← Produces an error as adding a vehicle with a parking location string containing characters not in the A-Z or 0-9 or . (period) ranges
A-CU57ABC-C-	← Produces an error because adding a vehicle with no parking location string
A-CU57ABC-M-BedfordSq.	← Adds a vehicle with reg number “CU57ABC” type motorcycle and parking location “BedfordSq.”
A-GR04XFB-B-BrownsLane	← Produces an error as there is a vehicle with the same reg number, type and parking location
Z-GR04XFB-BrownsLane	← Produces an error as it does not represent a known message format
ACU57ABC-V-BrownsLane	← Produces an error as it does not represent a known message format
A-GF23WSN- L- BrownsLane	← Produces an error as it does not represent a known message format because of the spaces

A python program for running on a host and conforming to the specification is provided with this assignment. You are welcome to modify this for your testing purposes. A version of this program will be used to test your implementations, although the messages might change.

2.5 Requirements

- You must implement your code using a Finite State Machine (FSM). Include a picture of this machine, listing the states and the state transitions in your documentation.
- Make sure you comment your code appropriately. Ideally code should be mostly self-explanatory through sensible choice of variable and function names and use of macros. However, there will be times when things are less comprehensive, so be sure to highlight these.
- In your documentation:
 - Write a description of your states, what is being waited for, and the actions taken during transition between states.
 - Write a description of the process that you have used for debugging in your code.
 - Write 200–300 words of reflection on your code. Include those things that don't work as well as you would like and how you would fix them.

2.6 Assessment

Achieving the above implementation is worth up to 70 marks, if properly documented (25 marks) and coded (45 marks).

3 Extension features

The remaining marks are distributed across the extension features. You may implement as many as you wish. Assume about 35% of each extension's marks will be based on any documentation.

The following extensions allow you to demonstrate more knowledge and in return are worth additional marks as indicated. If you have implemented any of these then the string BASIC sent after synchronisation must be replaced by a list of the extension names separated by commas and followed by a "Newline". For example:

UDCHARS,FREERAM,HCI,EEPROM,SCROLL

3.1 UDCHARS

This shows your ability to define your own character forms. [5 marks]

- Define some characters to replace the v and ^ with nicer looking arrows.

- Add to your documentation a section to discuss your UDCHARS implementation.

3.2 FREERAM

This shows your ability to read the free SRAM in an Arduino. [5 marks]

- Modify what happens when pressing the SELECT button. In addition to the ID number, display the amount of free SRAM.
- Add to your documentation a section to discuss your FREERAM implementation.

3.3 HCI

This shows your ability to select particular values from an array of values. [5 marks]

- When the user presses and releases the RIGHT button, the display must only display vehicles that have payment status PD (i.e. Paid). If none match this criterion, then display "NO PD VEHICLES".
- When the user presses and releases the LEFT button, the display must only display vehicles that have payment status NPD (i.e. Not Paid). If none match this criterion, then display "NO NPD VEHICLES".
- In either of the above two states, only repressing and releasing the opposite button returns to displaying all the vehicles.
- The v and ^ arrows should only appear based on the subset of vehicles to display.
- Add to your documentation a section to discuss your HCI implementation.

3.4 EEPROM

This shows your ability read/write from EEPROM. [5 marks]

- Store the vehicles reg number, type, payment status and parking location, entry and exit timestamps (if applicable) in the EEPROM so that they can survive power resets.
- Store current time to EEPROM every time a processed serial message causes any change in the system.
- Read from the EEPROM on start-up and find a mechanism for determining whether the values were written by you or simply left from before.
- Add to your documentation a section to discuss your EEPROM implementation. Also indicate if you foresee any problems with this approach regarding RAM size and EEPROM size and how these problems can be avoided.

3.5 SCROLL

This shows your ability to implement a more complicated FSM that can take time into account as well as listening to the Serial Monitor and button presses. [10 marks]

- If a vehicle's parking location string is too big (i.e. longer than 7 Characters) then add code to scroll it left at 2 characters per second, returning to the start when the full parking location has been displayed.
- All other functionalities should remain implemented as described.
- Add to your documentation a section to discuss your SCROLL implementation.

4 Hints

- SRAM is limited. You want to keep use of SRAM to only what really needs the SRAM. Make use of the FLASH memory for fixed strings.
- Be careful using the String class—this tends to use up the SRAM.
- On writing to EEPROM remember to use the correct calls to avoid writing when you don't need to.
- In your design of the FSM, consider what events your code will be waiting for. Think about button presses, button releases, something from the serial, timeouts.
- You can choose to interact with your program simply through the Serial Monitor, but it may help you to write some python host code to automate this. An example of this code is provided along with the coursework brief.

5 Implementation

1. You must use (at least one) Finite State Machines to implement this specification and these must be described in your final deliverable.
2. You must implement the protocols exactly as described. If you feel anything is not clear, then please ask.
We may use automated testing of your code.

6 Deliverables

This is an individual coursework. Similarity-detection software will be used on both the documentation and the source code as an initial check to see where collusion or copying have taken place. Code may be checked against other submissions this year, previous years and other sources as appropriate.

Your submission needs to include the following:

- Source code of your implementation: a single .ino file submitted to LEARN.
- A written report—submitted as a PDF along your code submission to LEARN.

6.1 Source code

Submit this as the actual .ino file, not copy-pasted into the report.

6.2 Written report

A template for the report is available on the Learn page in markdown, Word and LaTeX, your choice. But remember the final submission must be as a PDF. You must use the provided template—see LEARN.

Please note that other file formats will not be accepted, and simply renaming the report's extension to a PDF extension will be penalised and might be treated as a non-submission.

7 Further information

- You may only use the following libraries (as identified by their header files). You might not need to use all of these.

- Wire.h
- Adafruit_RGBLCDShield.h
- utility/Adafruit_MCP23017.h
- EEPROM.h
- avr/eeprom.h
- TimeLib.h
- TimerOne.h
- MemoryFree.h

- Your code must consist of a single .ino file, which needs to compile using the Arduino IDE. If the code fails to compile in this environment, then you risk failing the module.
- Your code must work with the Arduino Uno and LCD shield that we issued to you. In particular, if you develop your code on a different type of Arduino or with a different type of shield, you risk failing this module.
- We will use various types of software for code similarity detection to compare each submission to all others, to submissions from previous years, and to code from other sources. Please be reminded that this is an individual assessment, which means that group work is not permitted.

8 Grading scheme

Most marks will fall into a scale between 40 (scraped pass) and 80 (impressive). These are explained in the grading descriptors below.

Significantly fewer marks will be awarded if you do not follow the compulsory instructions, for example, not using the template, using other libraries than those specified above, using standard C++ code not designed for Arduino.

As a rough guide the following sections indicate what the expected mark ranges.

8.1 Code

Code covers the code itself and includes readability, sensible function names, use of functions and language features as well as whether the functionality is there.

- A (80+) - The project significantly exceeds the minimal specification in the number of features and in the technical challenge behind these features. All features have been realised at an excellent level. The project implementation carried out at a high-quality level.
- B (70) - The project significantly exceeds the minimal specification in the number of features or in the technical challenge behind these features. The project implementation has been executed well with some aspects at an excellent level.
- C (60) — The minimal feature set and some non-trivial extra features have been realised at an adequate level. The project implementation and testing is at an adequate level. The interface is easy to understand after some explanation.
- D (50) — The minimal feature set, and some minor extra features have been realised at a mostly adequate level. Enough has been done the overall achievement is mediocre.
- E (40) — The minimal feature set has been realised at a somewhat adequate level. The overall implementation and testing of the project is rudimentary.
- F (40) - Not even the minimum feature set working and nothing to make up for this. The higher end of this range would be used to reward partial attempts with some merit.

8.2 Documentation

Documentation will cover pictures and text in the report and comments in the code.

- A (80+) The documentation is beautiful and describes all aspects of the implementation well.
- B(70) The documentation is very good, including all details, but lacks in presentation and understanding.

- C(60) The documentation includes most of what is necessary, but misses some vital meaning.
- D(50) The documentation describes some aspects well, but these aspects only form some of what is required.
- E(40) The documentation is barely adequate.
- F(40) Something worse. No documentation will lead to a mark of zero.